A Deep Learning Pipeline for Human Activity Recognition

Bojan Dimovski[1], Ana Indovska[1], Marija Boshkoska[1], Tomi Nikoloski[1]

[1]Faculty of Electrical Engineering and Information Technologies

Outline

Outline

Goals for the project 3
Data collection 4
Preprocessing & Segmentation - IMU data 6
DL Architecture 9
Max-Pooling vs. Absmax-Pooling 10
“Small” model results - 6-Fold Cross-Validation 11
“Small” model results - External test dataset (HAR_jumping) 12
Sitting vs. standing - “Big” model - Preprocessing: Pressure sensor 13
An unsuccessful attempt 15
State modelling - Segmentation: successful version 16
“Big” model results 17
Future work 21

Goals for the project

Goal	Achieved?
Collect data for HAR from 30+ participants	✅✅ (42!!!)
Implement an entire preprocessing pipeline	✅
Finally understand how Git versioning works	✅🤦
Realize Pytorch is exhausting but infinitely more flexible	💀💀
Build custom pooling layers from scratch to fit domain	✅ (somewhat novel!)
Build a DL model that accurately classifies activities	✅✅
Utilize sensor fusion to differentiate sittting v. standing	📈📈
Workout dataset analysis & calorie estimation	Ana's BSc Thesis <3
Gait analysis	Tomi's BSc Thesis <3
Have fun 🤪	✅✅✅

Data collection

- Collect data from **42** participants in InnoFEIT and Emteq
- Supervise and annotate data for each participant individually
- Collected data for 5 tasks:
  - Everyday activities (our focus)
  - Reading
  - Transitions (standing up, sitting down, laying down...)
  - Workout (Ana's future work)
  - Gait (Tomi's future work)

Data collection

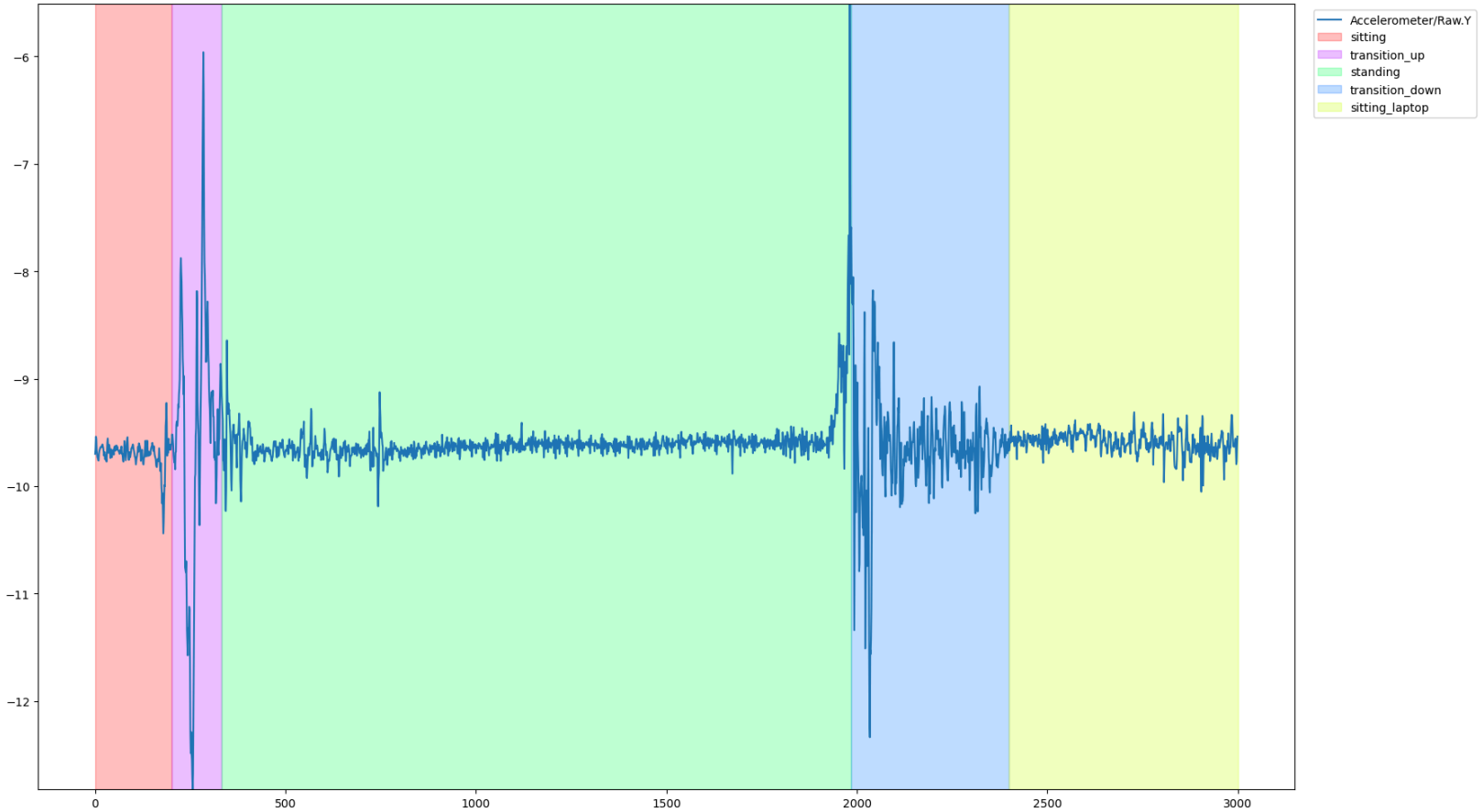
Imagine a month and a half of this...

Preprocessing & Segmentation - IMU data

- Accelerometer (X, Y, Z) ✅
- Gyroscope (X, Y, Z) ✅
- Magnetometer (X, Y, Z) ❌ (too sensitive to external conditions)
- Outlier isolation & smoothing - via rolling window average
- Low-Pass Butterworth filter (5th order) - $f_{\text{high\_cutoff}} = 4$ [Hz]
  - Attenuates high-frequency sensor noise, preserves low-frequency structural components of body movement.
- Euclidean norms: $A_{\text{mag}} = \sqrt{A_x^2 + A_y^2 + A_z^2}$, $G_{\text{mag}} = \sqrt{G_x^2 + G_y^2 + G_z^2}$
  - Same preprocessing applied as to individual channels
- $W = 4[\text{s}] = 200$ samples, $S = 1[\text{s}] = 50$ samples; 75% overlap

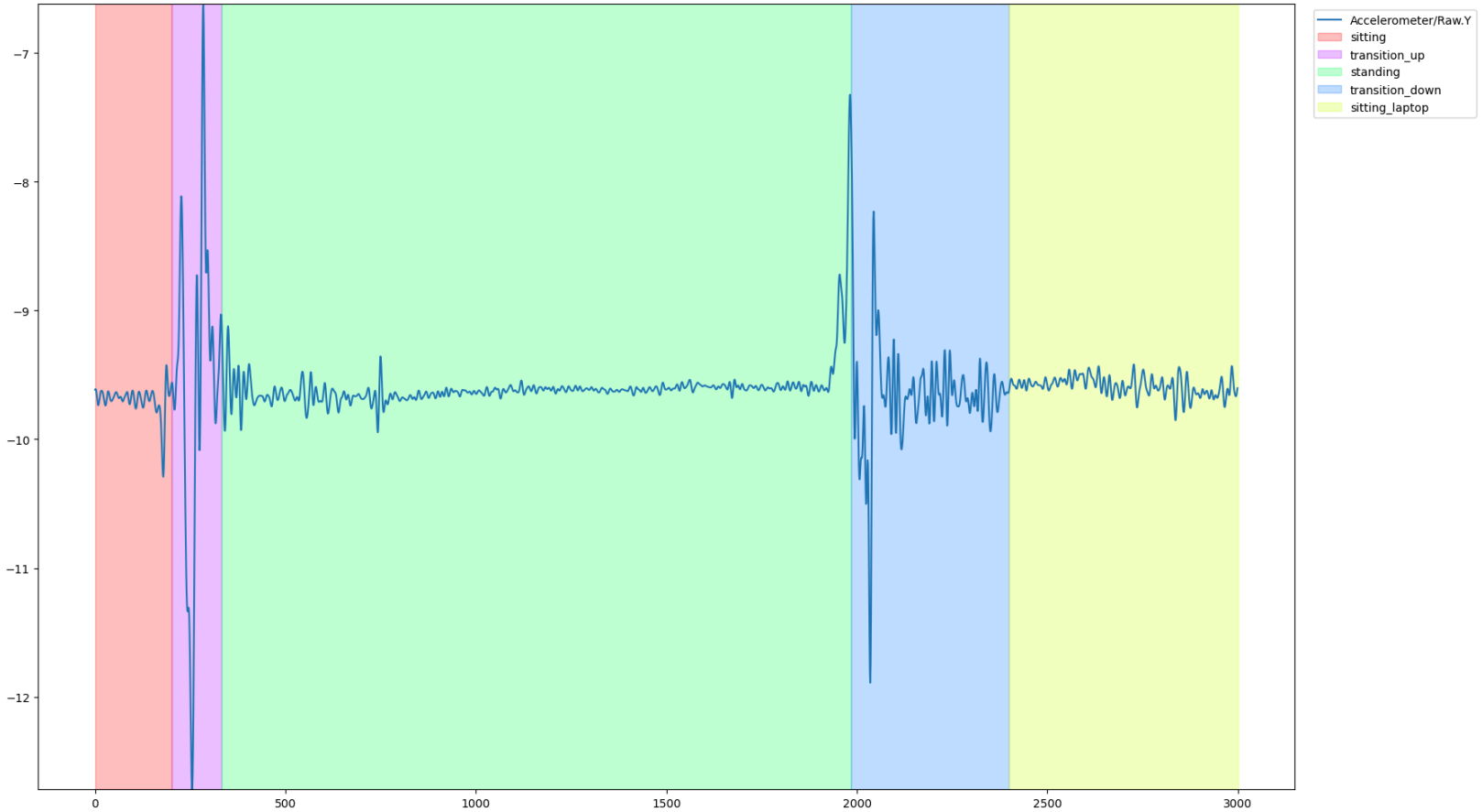
Effect of filtering

- Before filtering

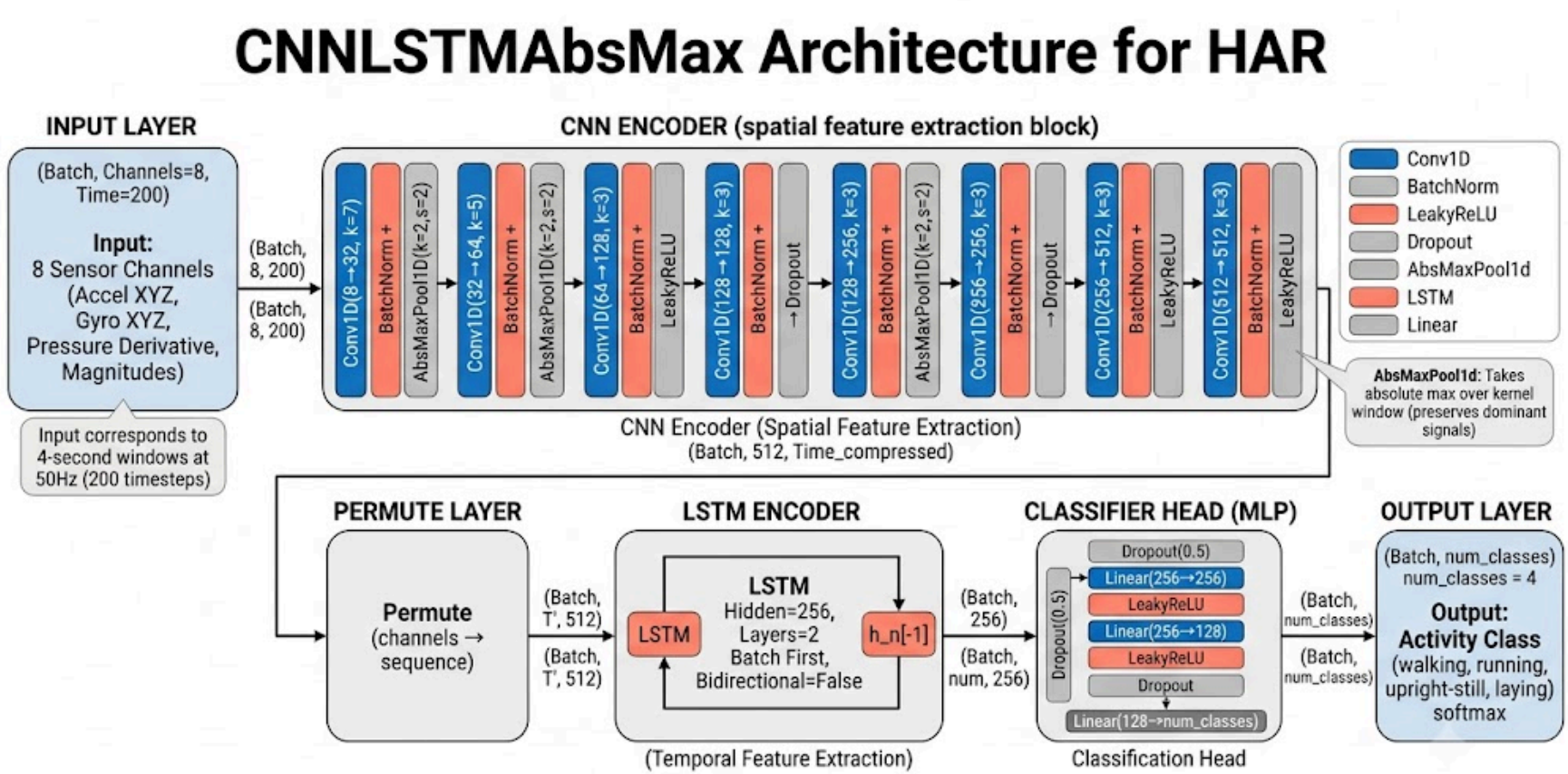


Effect of filtering

- After filtering

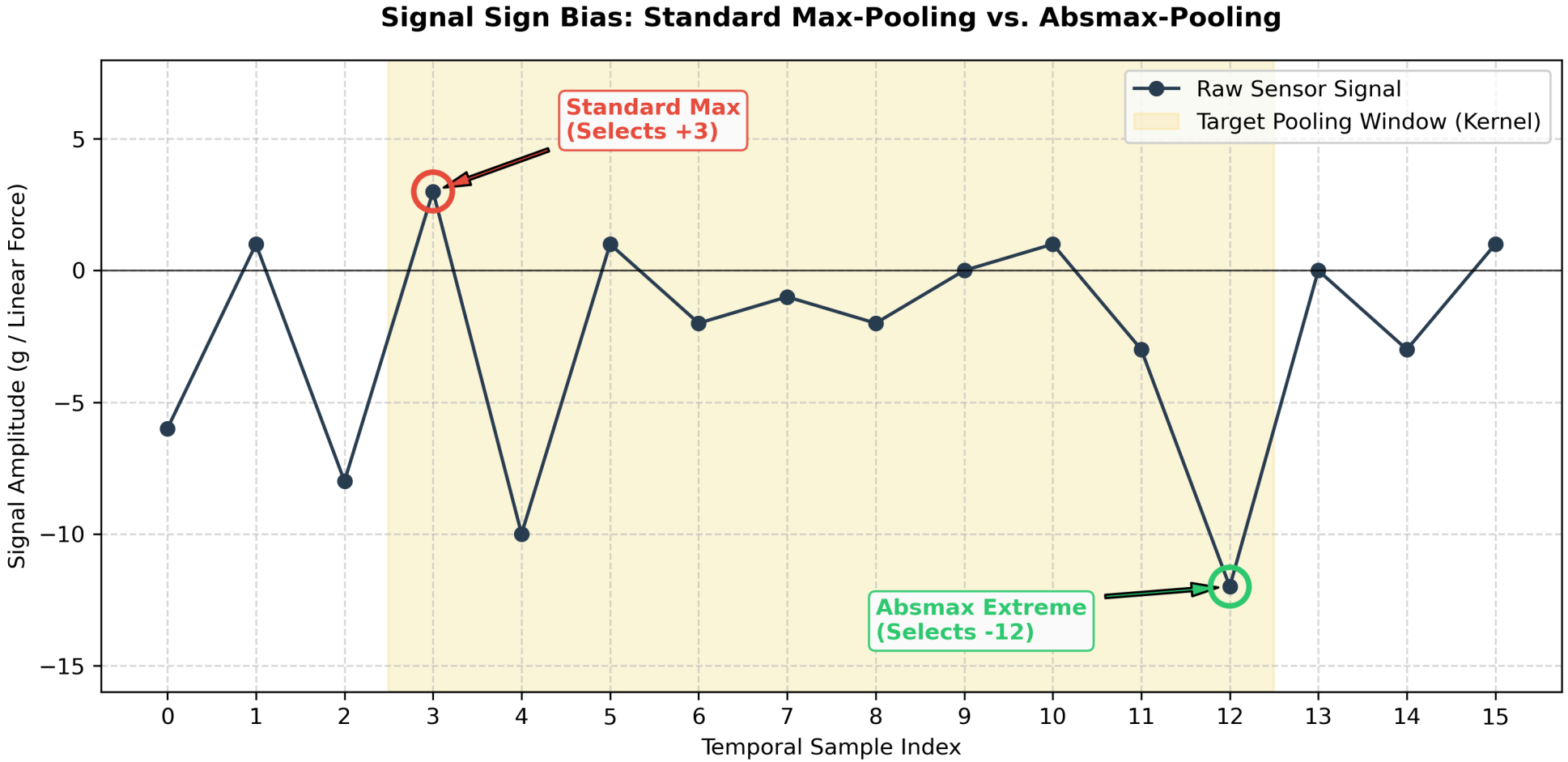


DL Architecture



Total trainable parameters:	[visible as CNNEncoder + LSTM + ClassifierHead]
Input:	8 channels × 200 timesteps
Sensor columns:	Accelerometer (X,Y,Z,Magnitude), Gyroscope (X,Y,Z,Magnitude), Pressure Derivative

Max-Pooling vs. Absmax-Pooling



“Small” model results - 6-Fold Cross-Validation

- Absmax-Pooling outperforms Max-Pooling
- Macro F_1 scores: $F_{1,\text{max}} = 0{,}9953$, $F_{1,\text{absmax}} = 0{,}9957$ - difference only in 4th d.p.

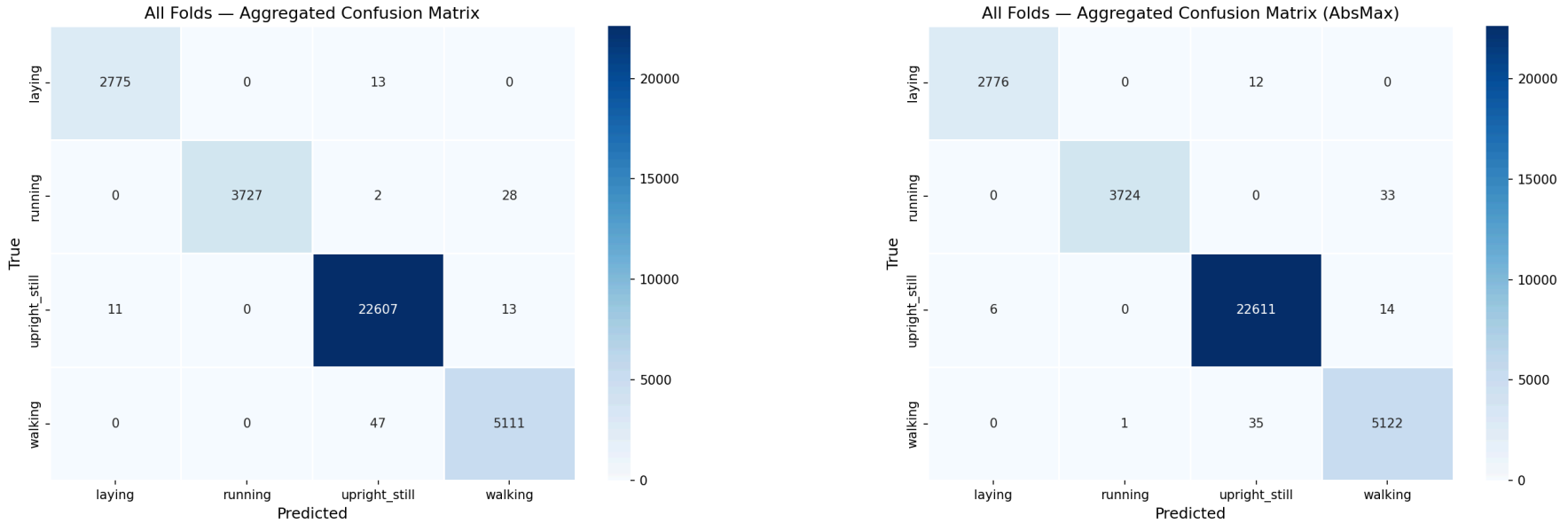


Fig. 1: Confusion Matrices for 6fCV - Max-Pooling (left) vs. Absmax-Pooling (right)

"Small" model results - External test dataset (HAR_jumping)

- Absmax-Pooling outperforms Max-Pooling yet again
- Macro F_1 scores: $F_{1,\text{max}} = 0{,}9931$, $F_{1,\text{absmax}} = 0{,}9967$ - difference only in 3rd d.p.

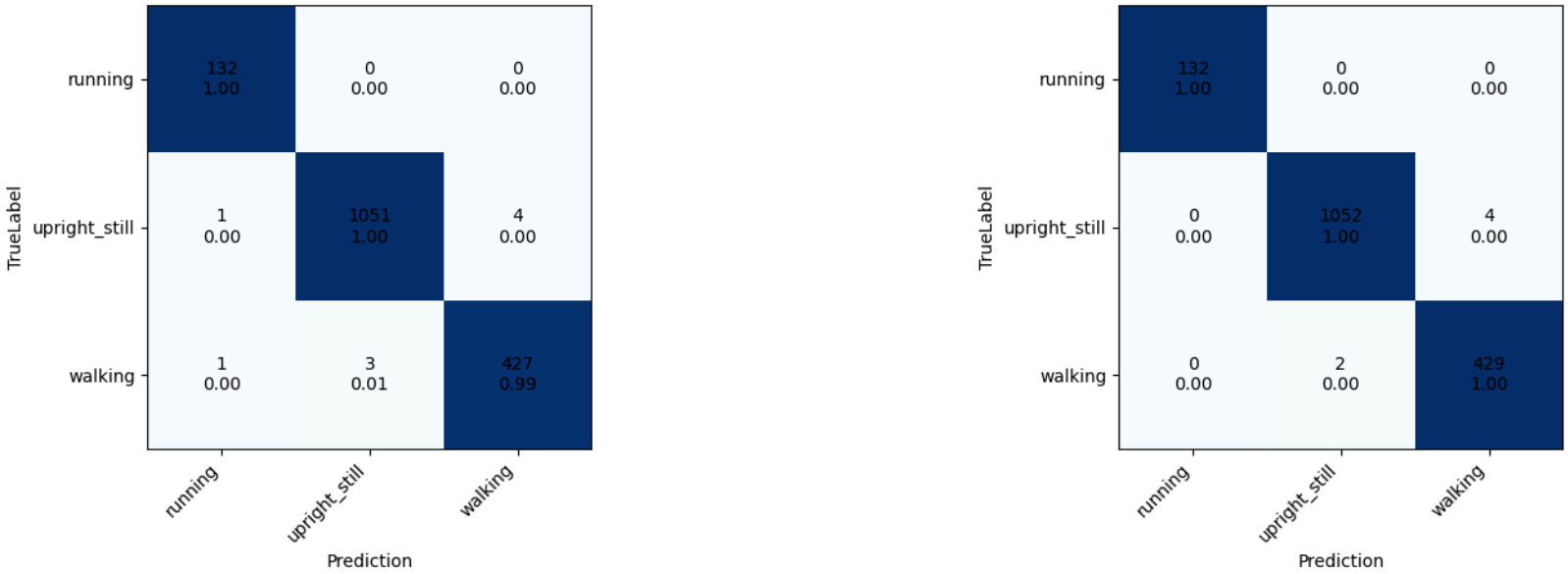
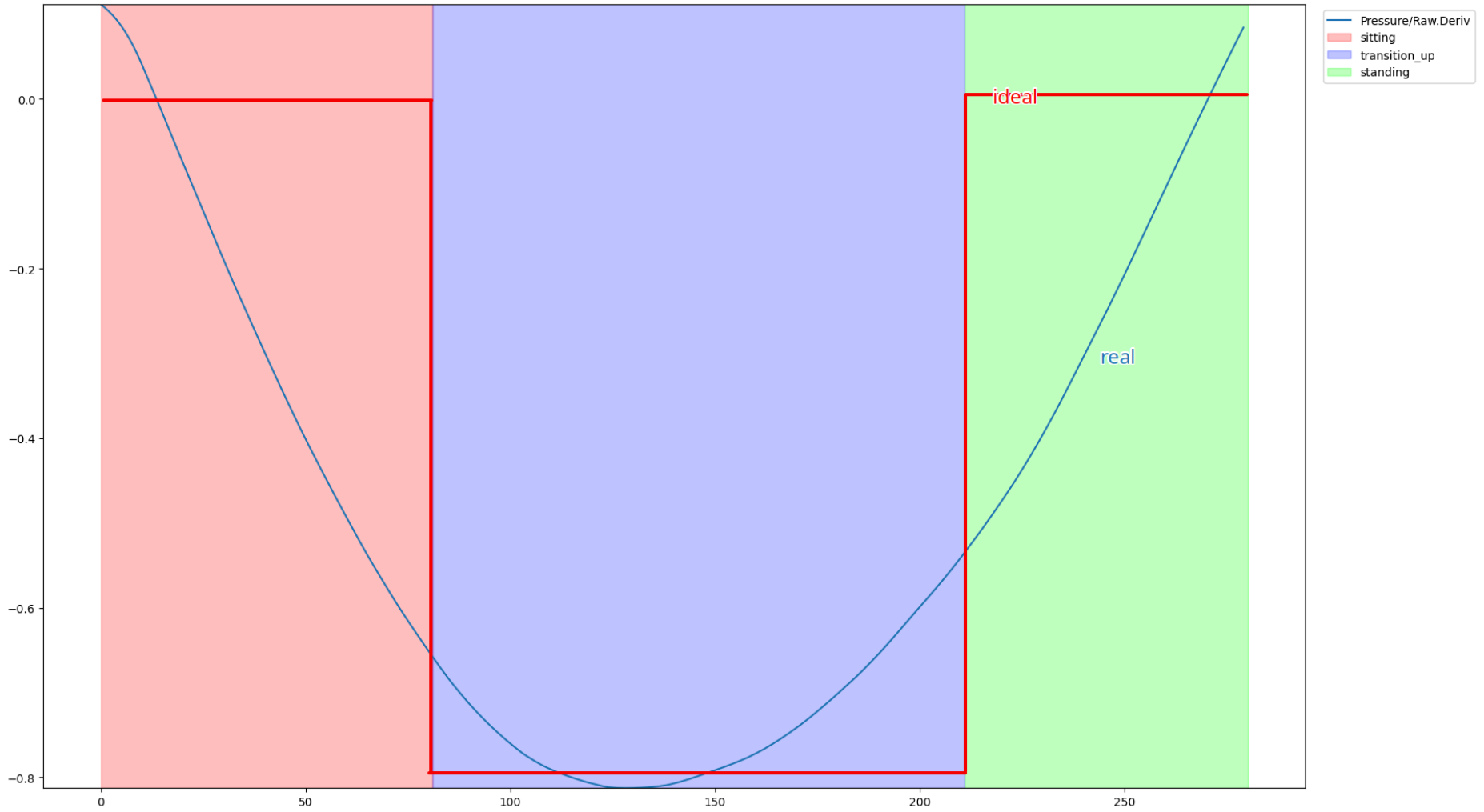


Fig. 2: Confusion Matrices for HAR_jumping - Max (left) vs. Absmax (right)

Sitting vs. standing - “Big” model - Preprocessing: Pressure sensor

- Leaving Max-Pooling behind
- Utilizing pressure sensor for height/altitude estimation
- Absolute pressure baselines fluctuate drastically between users due to varying body weights or baseline sensor fitting snugness.
- Use $\frac{dp}{dt}$:
  - Two-Stage Signal Smoothing: The raw pressure stream (`Pressure/Raw`) is first filtered using a localized median rolling window to eliminate micro-oscillations caused by shifting body weight during static poses.
  - Discrete Derivative: A Savitzky-Golay filter computes the first-order discrete derivative (`Pressure/Raw.Deriv`) scaled by the 50 Hz sampling frequency, capturing the instantaneous rate of change of force to normalize absolute baseline variations across different subjects and environmental altitudes.

Sitting vs. standing - “Big” model - Preprocessing: Pressure sensor



An unsuccessful attempt

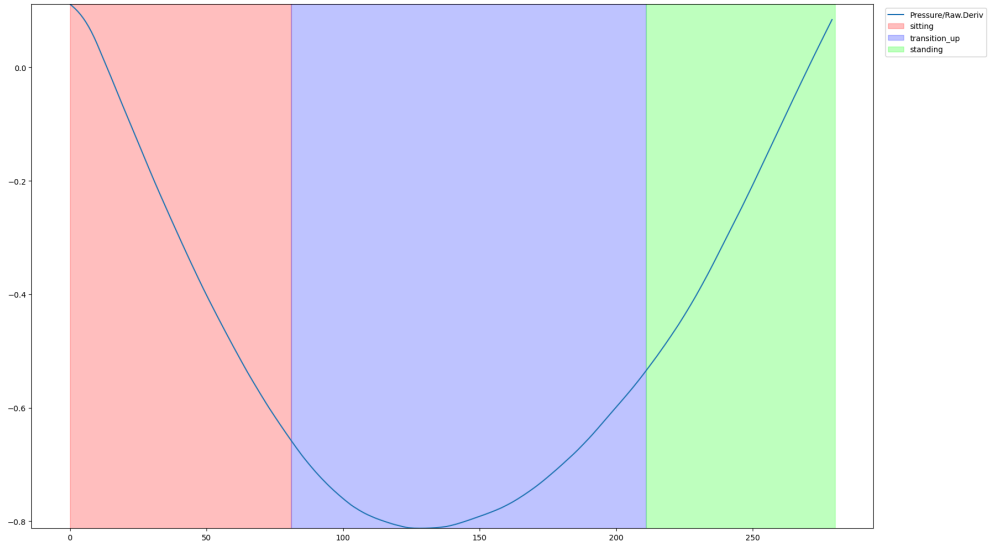
- As can be seen in the picture in the previous slide, the derivative of the pressure is ≈ 0 for non-transition classes, and negative ($\approx -0.7\left[\frac{\text{pressure\_units}}{\text{time}}\right]$) for the `transition_up` class. This means that adding the `Pressure/Raw.Deriv` as a channel won't be very informative for distinguishing sitting v. standing:

Activity Class	Precision	Recall	F_1-Score	Support
Sitting	0,6162	0,4737	0,5357	7407
Standing	0,7662	0,8561	0,8086	15754
Accuracy (all classes)	0,8221	0,8221	0,8221	37328
Macro Average (all 7)	**0,8924**	**0,8755**	**0,8822**	37328

- But, this gives us an idea!
  - The derivative of the pressure is not informative for the model to distinguish `sitting` and `standing` as separate classes, but it will be informative to distinguish additional `transition_up`/`transition_down` classes!

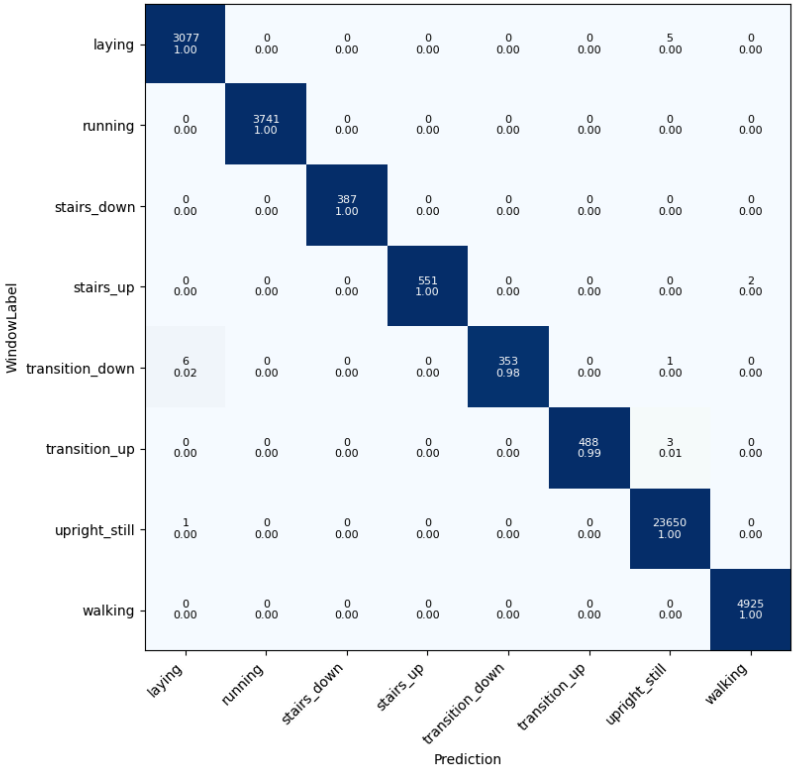
State modelling - Segmentation: successful version

- Model treats `sitting` & `standing` as 1 class: `upright_still`
- Model is trained on 2 additional classes: `transition_up`, `transition_down`
- Since a `transition_up`/`transition_down` segment lasts <1 second, a 4 second window is built from the signal space by backward and forward filling around the `transition_up`/`transition_down` segment so as to complete a 200-sample frame:



"Big" model results - Raw model outputs

- Raw model outputs showcase 0.98 recall for `transition_down`, 0.99 recall for `transition_up`!



"Big" model results - Postprocessing: Tracking state changes

- Use non-transition (static & dynamic classes, i.e. `sitting`, `walking`, `standing`, `running`) as anchor states
- Apply postprocessing informed by `transition_up` and `transition_down` triggers!
  - e.g. `upright_still`→`transition_up`→`walking` ⇒ `upright_still`=`sitting`
  - `upright_still[1]`→`transition_down`→`upright_still[2]` ⇒ ⇒`upright_still[1]`=`standing`, `upright_still[2]`=`sitting`
- Everytime a `transition_up`/`transition_down` window is detected, the next `upright_still`
- Results: Much better F_1 scores for `sitting` and `standing` classes.

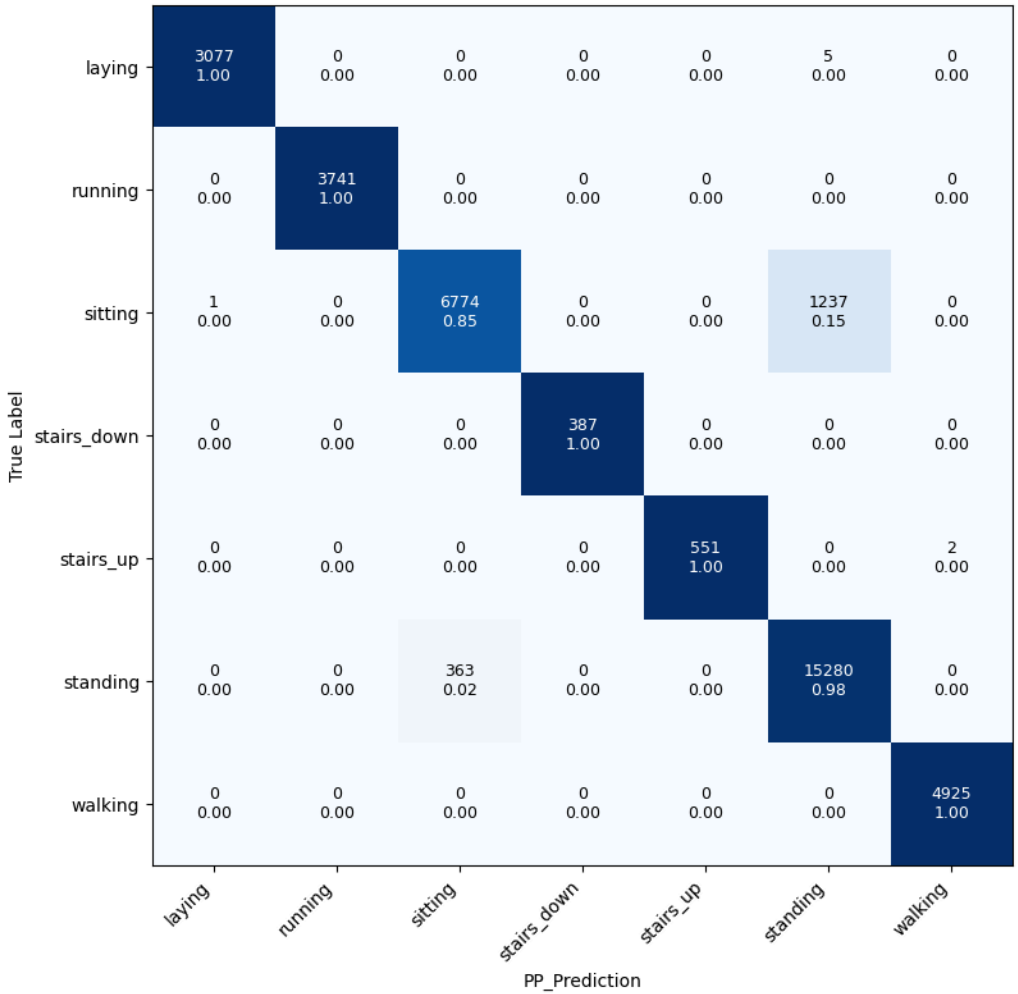
“Big” model results

TABLE I

DETAILED PERFORMANCE REPORT AFTER APPLYING STATE-TRANSITION TRACKING LOGIC.

Activity Class	**Precision**	**Recall**	**F_1-Score**	**Support**
Laying	0.9997	0.9984	0.9990	3,082
Running	1.0000	1.0000	1.0000	3,741
Sitting	0.9491	0.8455	0.8943	8,012
Stairs Down	1.0000	1.0000	1.0000	387
Stairs Up	1.0000	0.9964	0.9982	553
Standing	0.9248	0.9768	0.9501	15,643
Walking	0.9996	1.0000	0.9998	4,925
Accuracy	0.9558	0.9558	0.9558	36,343
Macro Average	**0.9819**	**0.9739**	**0.9773**	36,343
Weighted Average	0.9563	0.9558	0.9551	36,343

“Big” model results



Future work

- Dataset enrichment (augmentation, additional recording)
  - Sitting recall still bottleneck for model (0,85) due to missed `transition_down` triggers
- Absmax-Pooling significance requires statistical validation
- Additional testing
- Metabolic tracking and health metrics integration (Ana & Tomi <3)

Q&A

Thanks <3